Howard University
College of Engineering & Architecture
Department of Electrical Engineering & Computer Science

EECE 306-01: Fundamentals of Electromagnetics Laboratory

Dr. Su Yan, su.yan@howard.edu

Lab 1: Building the Toolbox, Constants and Vector Algebra

Objectives

By the end of this session you will have

- created the package skeleton that every remaining lab will build on,
- implemented and documented the first eight functions of the toolbox,
- run the provided test harness and seen your checks pass.

1. What this course produces

This laboratory is not a sequence of twelve unrelated programming exercises. Over the semester your team builds one thing, a working electrostatic and magnetostatic field solver. One module is added each week and submitted that week, so the code is reviewed as it grows rather than all at once at the end.

Each week also adds a few tests. Those tests are never retired. In Week 11 you still run the tests written in Week 1, and they must still pass. Adding new code that breaks working older code is the most common way a growing program goes wrong, and keeping the whole test suite running is how that gets caught the same week it happens instead of in the final week.

By the last week your toolbox will contain 53 functions. It will compute $\vec{\boldsymbol{E}}$, V, $\vec{\boldsymbol{D}}$, $\vec{\boldsymbol{H}}$ and $\vec{\boldsymbol{B}}$ for arbitrary charge and current distributions, solve Poisson's equation on a grid with dielectric interfaces, and return capacitance, resistance, inductance, force and stored energy. The closing weeks are not new assignments, they are demonstrations that the instrument you built actually works on problems the textbook cannot solve in closed form.

Two documents govern the semester. The *API Specification* fixes the name, arguments, return values and units of every function. It is a contract and it does not change. This handout, and the eleven that follow, tell you what to build each week and how to prove that it is right.

Why the interface is fixed

The toolbox is assessed against the specification exactly as written, so a renamed function counts as missing. More importantly, building against a specification you did not choose is the most transferable skill in this course. Every engineer does it, every day.

2. Setting up

This course runs on GNU Octave, which is free and open source and installs on Windows, macOS and Linux. Start it by typing `octave` in a terminal. Every function you write also runs unchanged under MATLAB, so a student who already has MATLAB access may use it, but nothing in this course needs a license. Submissions are assessed on Octave, so test on Octave before you submit.

Create the folder structure below.

The leading plus sign in `+em` is not part of the name, it is package syntax that MATLAB and Octave both understand. A folder whose name begins with `+` is a *package*, which is how both environments group functions under a namespace. The folder is called `+em` on disk, the package is called `em` in code, and a function `mag.m` sitting inside `+em/+vec/` is called as `em.vec.mag(A)`.

Packages exist so that names cannot collide. Octave and MATLAB both provide a built-in `cross` and a built-in `angle`, and this toolbox defines its own. Without a package your `angle.m` would shadow the built-in for the entire session and break unrelated code. Inside a package, `em.vec.angle` and the built-in `angle` coexist with no conflict. It also means the whole toolbox is one self-contained folder you can hand to somebody, rather than forty loose files with names like `em_vec_mag.m`.

```
EECE306-Toolbox/
  +em/
    +const/
    +vec/
    +test/
  tests/
  runTests.m
  README.md
```

The simplest habit is to start Octave inside the toolbox folder, since the current directory is always searchable and no path work is needed,

```
$ cd EECE306-Toolbox
$ octave
```

Working from anywhere else, add the toolbox root, meaning the folder that contains `+em`, to the path,

```
>> addpath('~/EECE306-Toolbox') % right, the folder containing +em
>> addpath(pwd) % same idea when already inside it
```

but do **not** put `+em` itself on the path, `addpath('EECE306-Toolbox/+em')` is a common first-day error and it breaks the namespace, since packages are found through their parent folder. Verify with

```
>> what em
```

which should list the package contents once the first functions exist.

`runTests.m` is provided to you on Canvas. Do not modify it. It finds every file matching `tests/test_lab*.m`, runs it, and prints a pass and fail summary.

3. Part I. Physical constants

Implement the four functions of `em.const`. Each takes no argument and returns one scalar.

Function	Quantity	Requirement
`em.const.eps0()`	ε_0	$8.8541878128 \times 10^{-12}$ F/m
`em.const.mu0()`	μ_0	$4\pi \times 10^{-7}$ H/m
`em.const.c0()`	c_0	computed as $1/\sqrt{\mu_0\varepsilon_0}$
`em.const.eta0()`	η_0	computed as $\sqrt{\mu_0/\varepsilon_0}$

The last two must be *computed* from the first two, not typed in. Deriving them keeps a single

source of truth, so that a change to one constant propagates everywhere and no two copies can disagree. This is the same reason production code defines a constant once and refers to it, rather than repeating its value.

4. Part II. Vector algebra

Implement the four functions of `em.vec`. Each accepts an `Nx3` array, one vector per row, and returns `Nx1` or `Nx3`.

4.1 `em.vec.mag` and `em.vec.fromTo`

$$|\vec{\boldsymbol{A}}| = \sqrt{A_x^2 + A_y^2 + A_z^2}, \qquad \vec{\boldsymbol{d}} = \vec{\boldsymbol{Q}} - \vec{\boldsymbol{P}}$$

Both are one line if you use `sum(A.^2,2)` rather than indexing components. Write them vectorized from the start. From Lab 5 onward your functions must handle arrays with 10^4 rows in a couple of seconds, and a loop will not keep up.

4.2 `em.vec.unit`

$$\hat{A} = \frac{\vec{\boldsymbol{A}}}{|\vec{\boldsymbol{A}}|}$$

A zero vector has no direction, and dividing it by its own zero magnitude gives `NaN` in every component, which is the right answer. No special case is needed.

4.3 `em.vec.angle`

The interior angle between two vectors,

$$\theta = \arccos\left(\frac{\vec{\boldsymbol{A}} \cdot \vec{\boldsymbol{B}}}{|\vec{\boldsymbol{A}}|\,|\vec{\boldsymbol{B}}|}\right)$$

Implement it directly. Verify that it returns $\pi/2$ for a pair of perpendicular vectors, 0 for a parallel pair, and π for an antiparallel pair. Because the argument of `acos` can stray just past ± 1 through rounding, clamp it to $[-1, 1]$ with `max(-1, min(1, x))` before the call, so that a valid pair of vectors never produces a complex result.

5. Part III. The provided checker

The starter ships with one more ready made tool beside `runTests`, the comparison function `em.test.assertClose(act expected, tol, msg)`. Do not modify it. It stays silent when the two values agree to within the tolerance and stops with an error message when they do not, which is exactly what a test file needs. Floating point results never match a formula digit for digit, so `==` is the wrong comparison and this function is the right one.

Usage is one line per check,

```
em.test.assertClose(em.vec.mag([3 4 0]), 5, 1e-12, 'mag 3-4-5');
```

`tol` defaults to 10^{-10} when omitted, and `msg` labels the check so a failure says which line it came from. When the expected value is zero the comparison reads as "smaller than `tol` in magnitude," which is how a check like "this component should vanish" is written from Lab 3 onward.

Required tests

Write `tests/test_lab01.m` containing at least these checks.

1. `em.const.c0()` agrees with $1/\sqrt{\mu_0 \varepsilon_0}$, and equals about 3.00×10^8 m/s.
2. `em.const.eta0()` equals about 377 Ω.
3. `em.vec.mag([3 4 0])` equals 5 exactly.
4. `em.vec.unit([3 4 0])` equals `[0.6 0.8 0]`.
5. `em.vec.unit([0 0 0])` returns all `NaN`.
6. `em.vec.angle([1 0 0],[0 1 0])` returns $\pi/2$.
7. `em.vec.angle([1 0 0],[-1 0 0])` returns π and not a complex number.
8. All four `em.vec` functions accept a `100x3` input and return the correct shape.

`runTests` must report all of these passing before you leave.

6. Part IV. Documentation

Every function carries a help block. This one is graded, on a random sample, by running `help` on your functions.

```
function u = unit(A)
%UNIT Unit vector of each row of A.
% U = EM.VEC.UNIT(A) returns the Nx3 array of unit vectors of the
% Nx3 input A. A zero row produces a row of NaN.
%
% Example:
% em.vec.unit([3 4 0]) % returns [0.6 0.8 0]
%
% See also EM.VEC.MAG.
```

Start the `README.md` as well. For now it needs the team name, the member names, and a one line description of each module you have built.

Deliverables

Submit through Canvas by the start of the next lab session.

1. **Toolbox snapshot**, the whole `EECE306-Toolbox` folder as a single zip, named `TeamNN_Lab01.zip`.
2. **Lab notebook**, a published script exported to PDF, three to five pages. It must show each function working on the checks above and the output of `runTests`.
3. **Test output**, the printed summary from `runTests`, pasted into the notebook.

A lab notebook is an ordinary commented script, `Lab01_notebook.m`, run through the `publish` command that both Octave and MATLAB provide. Calling `publish('Lab01_notebook.m')` executes the script and writes one document that interleaves your code, its printed output, and every figure. In Octave this produces an HTML file, which you print to PDF from the browser. The template `Lab01_notebook.m` is on Canvas. Fill it in, do not start from a blank file.

Looking ahead

Next week you build the coordinate conversion module, eight functions. It contains the single most common conceptual error in this subject, the confusion between converting a *point* and converting the *components of a vector that lives at a point*. Read Section 2.3 and 2.4 of Wentworth before you come.